

Plantillas abstractas para agentes inteligentes

Notas de clase — patrones de implementación para agentes reactivos simples y agentes basados en modelo

Jairo
Ingeniería de Sistemas, Universidad del Magdalena

Septiembre de 2026

Resumen

Estas notas presentan, en forma de plantilla reutilizable, la estructura de implementación común a los ejercicios de agentes inteligentes de un curso introductorio de Inteligencia Artificial. La exposición parte de los conceptos compartidos por todo agente —percepción, acción y regla condición-acción— y a partir de ellos deriva dos plantillas concretas: la del agente reactivo simple, que decide únicamente con la percepción del instante, y la del agente basado en modelo, que mantiene un estado interno actualizado por sus propias acciones. Cada plantilla se acompaña de la definición en prosa de sus componentes y del procedimiento para instanciarla a partir del enunciado de un ejercicio. El documento es deliberadamente no matemático: su objeto es un patrón de implementación en Python, no un desarrollo con enunciados demostrables. Se asume como prerequisite el manejo de funciones, diccionarios y estructuras condicionales en Python. La taxonomía de agentes que organiza estas notas es la establecida por Russell y Norvig [1].

Índice

1	Conceptos comunes	2
1.1	Percepción, acción y reglas	2
1.2	La función de percepción	2
1.3	La diferencia entre los dos tipos de agente	3
2	Agente reactivo simple	3
2.1	Componentes	3
2.2	Plantilla	3
2.3	Modo de uso	4
3	Agente basado en modelo	4
3.1	Componentes	5
3.2	Plantilla	5
3.3	Modo de uso	7
4	Criterio de elección y errores frecuentes	8
4.1	Errores frecuentes	8
4.2	Síntesis	8

1 Conceptos comunes

Los dos tipos de agente que cubren estas notas se implementan con piezas distintas, pero descansan sobre el mismo vocabulario. Antes de presentar cualquiera de las dos plantillas conviene fijar ese vocabulario una sola vez: qué es lo que el agente observa, qué es lo que el agente hace, y cómo se enlaza lo uno con lo otro. Las secciones siguientes reutilizan estos términos sin volver a definirlos.

1.1 Percepción, acción y reglas

Todo agente opera dentro de un entorno que no controla por completo. De ese entorno recibe información, y sobre ese entorno ejecuta acciones. La primera tarea al resolver un ejercicio es delimitar con precisión ambos conjuntos, porque el enunciado siempre los declara de forma explícita—normalmente bajo los encabezados «Percepciones» y «Acciones» o «Reglas»— y el resto de la implementación se deduce de ellos.

Definición 1.1 (Percepción). La percepción es la información que el agente recibe del entorno en un instante dado. Se organiza como una colección ordenada de valores, uno por cada variable observable que el enunciado declara relevante para la decisión.

Definición 1.2 (Acción). Una acción es cada una de las respuestas que el agente puede ejecutar sobre el entorno. El conjunto de acciones posibles está fijado por el enunciado y es cerrado: el agente no inventa acciones nuevas, solo selecciona una de las disponibles.

Definición 1.3 (Regla condición-acción). Una regla condición-acción es la traducción directa de un enunciado «SI ... ENTONCES ...» del ejercicio: una condición evaluable, junto con la acción que el agente ejecuta cuando esa condición se cumple. El comportamiento de un agente queda determinado por una secuencia ordenada de reglas de este tipo.

Observación 1.4. El orden de las reglas es parte de su significado. Cuando dos condiciones pueden cumplirse simultáneamente ante una misma percepción, la que aparezca primero es la que decide. Reordenar las reglas al implementarlas, aunque sea por comodidad, puede alterar el comportamiento del agente sin producir ningún error visible.

1.2 La función de percepción

Ambas plantillas comienzan por la misma función. Su papel es empaquetar las variables observadas en una sola estructura, de modo que la lógica del agente reciba un único argumento y no una lista de parámetros que cambia de ejercicio en ejercicio. Es una función deliberadamente trivial: no decide nada, no valida nada y no modifica nada.

```
def percepcion(entrada_0, entrada_1, ..., entrada_n) -> tuple:
    return entrada_0, entrada_1, ..., entrada_n
```

La anotación de retorno se deja como `tuple` porque el número y el tipo de las entradas varían de un ejercicio a otro. Al instanciar la plantilla se reemplaza por `tuple[tipo_0, tipo_1, ..., tipo_n]` con los tipos reales de cada entrada, por ejemplo `tuple[bool, str]` para un agente que observa una variable booleana y una cadena.

1.3 La diferencia entre los dos tipos de agente

La distinción que organiza el resto de estas notas es una sola: qué información tiene disponible el agente en el momento de decidir. El agente reactivo simple dispone únicamente de la percepción actual; cualquier cosa que haya ocurrido antes es, para él, inexistente. El agente basado en modelo dispone además de un estado interno que resume lo ocurrido hasta ese momento, y que sus propias acciones han ido modificando.

De ahí se sigue el criterio práctico para clasificar un ejercicio: si dos percepciones idénticas deben producir siempre la misma acción, el agente es reactivo simple; si dos percepciones idénticas pueden producir acciones distintas según lo que haya pasado antes, el agente necesita un modelo. La Sección 4 desarrolla este criterio con más detalle, una vez presentadas ambas plantillas.

2 Agente reactivo simple

Un agente reactivo simple selecciona su acción exclusivamente a partir de la percepción del instante presente. No conserva registro de percepciones anteriores ni de las acciones que él mismo ejecutó, de modo que su comportamiento es enteramente predecible a partir de la entrada actual. Es el caso más frecuente en los ejercicios cuyo enunciado describe sensores y umbrales: un termostato, un sensor de presencia, un detector de obstáculos.

2.1 Componentes

La plantilla tiene dos piezas: la función de percepción ya presentada en la Sección 1, y la lógica del agente, que es donde vive todo el comportamiento.

Definición 2.1 (Lógica del agente reactivo simple). La lógica del agente reactivo simple es la función que recibe una percepción y evalúa las reglas condición-acción en el orden en que fueron declaradas, devolviendo la acción asociada a la primera condición que se cumple. Si ninguna condición se cumple y el enunciado especifica una acción por defecto, esa es la acción devuelta; si el enunciado garantiza que las reglas cubren todos los casos posibles, la rama por defecto no es necesaria.

2.2 Plantilla

Los dos bloques conforman un solo archivo; se separan aquí para leer cada pieza junto a la explicación de su papel.

a) Percepción. Empaqueta las variables observadas, tal como se definió en la Sección 1.

```
def percepcion(entrada_0, entrada_1, ..., entrada_n) -> tuple:
    return entrada_0, entrada_1, ..., entrada_n
```

b) Lógica del agente. Contiene las reglas del enunciado, encadenadas en su orden original.

```
def logica_agente(percepcion_actual: tuple) -> str:
    entrada_0, entrada_1, ..., entrada_n = percepcion_actual

    if condicion_1(entrada_0, entrada_1, ..., entrada_n):
        return accion_1
    elif condicion_2(entrada_0, entrada_1, ..., entrada_n):
        return accion_2
    ...
    elif condicion_k(entrada_0, entrada_1, ..., entrada_n):
        return accion_k
    else:
        return accion_default # solo si el enunciado no garantiza reglas exhaustivas
```

Observación 2.2. Las ramas se encadenan con `elif` y no con `if` independientes. Con `if` sueltos, varias reglas podrían ejecutarse ante una misma percepción y el agente dejaría de seleccionar una única acción, que es precisamente lo que la definición exige. El uso de `return` dentro de cada rama produce el mismo efecto excluyente, pero encadenar con `elif` lo hace explícito para quien lee el código.

2.3 Modo de uso

1. Identificar las entradas listadas en la sección «Percepciones» del enunciado y nombrarlas `entrada_0, ..., entrada_n` en el mismo orden en que el enunciado las presenta.
2. Reemplazar la anotación `tuple` por los tipos concretos de esas entradas.
3. Trasladar cada regla «SI... ENTONCES...» a un par condición-acción, conservando el orden exacto en que el enunciado las enumera.
4. Traducir cada condición en una expresión booleana sobre las entradas, dentro del `if/elif` correspondiente.
5. Verificar si las reglas cubren todos los casos posibles. Si no los cubren, conservar la rama `else` con la acción por defecto que el enunciado especifique; si los cubren, puede omitirse.
6. Construir al menos un caso de prueba por regla, diseñado para no disparar ninguna regla anterior en el orden del `if/elif`: solo así la prueba confirma que esa regla en particular —y no una anterior que la enmascara— es la que produce la acción observada.

3 Agente basado en modelo

Hay ejercicios en los que la percepción actual no alcanza para decidir. Si una biblioteca debe evitar que un estudiante reserve dos salas, el agente necesita saber si ese estudiante ya reservó antes; si un hospital debe evitar enviar la misma ambulancia a dos emergencias, necesita saber cuáles unidades ya despachó. En ambos casos la información decisiva no está en el sensor: está en lo que ocurrió y el agente registró. Un agente basado en modelo es el que mantiene ese registro y lo consulta al decidir.

3.1 Componentes

A las dos piezas del agente reactivo simple se suman tres: la estructura que guarda el estado interno, las funciones que lo modifican y la función que lo muestra.

Definición 3.1 (Modelo o estado interno). El modelo es la estructura de datos donde el agente registra la información del entorno que necesita recordar entre una percepción y la siguiente. Se implementa como un diccionario cuyas claves son las variables que el agente debe rastrear, y cuyos valores son las estructuras de datos apropiadas para cada una: un entero para un contador, un conjunto para una colección sin duplicados, un diccionario anidado para una correspondencia entre entidades.

Definición 3.2 (Función de creación del modelo). La función de creación del modelo construye y devuelve el estado interno en su configuración inicial, es decir, la que corresponde al momento anterior a cualquier percepción. Aislar esta construcción en su propia función permite reiniciar el agente sin duplicar la definición del estado.

Definición 3.3 (Funciones auxiliares de actualización). Las funciones auxiliares de actualización son las que modifican el modelo como consecuencia de una acción del agente. Cada una encapsula una modificación con significado propio dentro del dominio del problema —registrar una asignación, liberar un recurso, incrementar un contador— y recibe el modelo como parámetro para modificarlo en el lugar.

Definición 3.4 (Lógica del agente basado en modelo). La lógica del agente basado en modelo es la función que recibe una percepción y el modelo, evalúa las reglas condición-acción consultando ambos, y ejecuta la acción correspondiente a la primera condición que se cumple. Ejecutar la acción significa invocar las funciones auxiliares que actualizan el modelo y comunicar el resultado; la función no devuelve un valor.

Definición 3.5 (Función de presentación del estado). La función de presentación del estado recorre el modelo y muestra su contenido actual. No decide ni modifica nada: existe para inspeccionar qué ha registrado el agente hasta el momento, lo que permite comprobar que las actualizaciones ocurrieron como se esperaba.

3.2 Plantilla

La plantilla se presenta por bloques, en el orden en que conviene escribirlos. Los cinco bloques conforman un solo archivo; se separan aquí para que cada pieza pueda leerse junto a la explicación de su papel.

a) **Creación del modelo.** Las claves son las variables que el agente debe recordar; los valores, la estructura de datos adecuada a cada consulta.

```
def crear_modelo() -> dict:
    modelo = {
        "variable_aprendizaje_0": estructura_de_datos_0(),
        ...
        "variable_aprendizaje_k": estructura_de_datos_k()
    }
    return modelo
```

b) **Percepción.** Idéntica a la del agente reactivo simple.

```
def percepcion(entrada_0, entrada_1, ..., entrada_n) -> tuple:
    return entrada_0, entrada_1, ..., entrada_n
```

c) **Funciones auxiliares.** Las de actualización modifican el modelo y no devuelven valor; las de consulta lo interrogan y sí devuelven uno.

```
def auxiliar_1(argumentos, modelo: dict) -> None:
    # modifica modelo["variable_aprendizaje_i"]
    # como consecuencia de una accion
    ...

def auxiliar_m(argumentos, modelo: dict):
    # las auxiliares de consulta SI devuelven un valor
    # (por ejemplo: cuantos cupos quedan, si un recurso esta libre)
    ...
```

d) **Lógica del agente.** Las condiciones consultan la percepción y el modelo; el cuerpo de cada rama invoca las auxiliares que correspondan.

```
def logica_agente(percepcion_actual: tuple, modelo: dict) -> None:
    entrada_0, entrada_1, ..., entrada_n = percepcion_actual

    if condicion_1(entrada_0, ..., entrada_n, modelo):
        auxiliar_i(argumentos, modelo)
        print(mensaje_accion_1)
    elif condicion_2(entrada_0, ..., entrada_n, modelo):
        auxiliar_j(argumentos, modelo)
        print(mensaje_accion_2)
    ...
    elif condicion_k(entrada_0, ..., entrada_n, modelo):
        print(mensaje_accion_k) # una regla puede no modificar el modelo
    else:
        print(mensaje_accion_default)
```

e) **Presentación del estado.** Se recorre el diccionario con un ciclo para que siga funcionando si más adelante se agrega una variable al modelo.

```
def mostrar_estado_actual(modelo: dict) -> None:
    for clave, valor in modelo.items():
        print(f"{clave}: {valor}")
```

Observación 3.6. Las condiciones de este agente pueden consultar el modelo además de la percepción, y eso es justamente lo que lo distingue del reactivo simple. Una condición que solo mire las entradas y nunca el modelo indica que esa regla no necesitaba estado interno; si todas las reglas son así, el ejercicio no era de este tipo.

Observación 3.7. No toda regla modifica el modelo. Rechazar una solicitud, informar una asignación previa o advertir una inconsistencia son acciones legítimas que dejan el estado intacto. Lo que sí debe cumplirse es lo contrario: toda modificación del modelo debe ocurrir dentro de una rama, como consecuencia de una regla, y nunca antes de evaluarlas.

Observación 3.8. La lógica del agente no devuelve valor porque su efecto es la modificación del estado y la comunicación del resultado por pantalla. Una variante igualmente válida devuelve además una cadena con la acción seleccionada, lo que facilita escribir pruebas automáticas que verifiquen la decisión sin depender de lo impreso. Si el enunciado exige una u otra forma, se sigue el enunciado.

Nota 3.9. La expresión «variable de aprendizaje», usada en la plantilla por continuidad con el vocabulario del curso, designa aquí una variable de estado interno: algo que el agente registra y consulta. No debe confundirse con el aprendizaje en el sentido de los agentes que aprenden, donde lo que cambia con la experiencia es la propia regla de decisión y no solo los datos que esta consulta [1]. En las plantillas de estas notas las reglas son fijas.

3.3 Modo de uso

1. Identificar en el enunciado qué información debe recordarse entre percepciones. Suele aparecer explícita bajo un encabezado como «Estado interno inicial»; si no aparece, se deduce de las reglas que mencionan el pasado («si ya tiene una reserva», «si la ambulancia está ocupada»).
2. Elegir la estructura de datos de cada variable según cómo se consulte: un conjunto si la pregunta es de pertenencia, un diccionario si es una correspondencia entre entidades, un entero si es un conteo. Escribir `crear_modelo` con esas variables en su valor inicial.
3. Distinguir las cantidades que se almacenan de las que se derivan. Un límite fijo, como una capacidad máxima, es un dato constante del modelo; los cupos disponibles se calculan a partir de él y del estado actual, y no deben guardarse como una segunda variable que haya que mantener sincronizada.
4. Escribir las entradas de la percepción igual que en el agente reactivo simple.
5. Escribir una función auxiliar por cada modificación con significado propio en el dominio. Mantenerlas pequeñas y de una sola responsabilidad: si una auxiliar hace dos cosas, probablemente son dos auxiliares.
6. Trasladar las reglas del enunciado al `if/elif` de la lógica, conservando su orden. En las condiciones, consultar el modelo donde la regla lo requiera; en el cuerpo de cada rama, invocar las auxiliares que corresponda.

7. Escribir `mostrar_estado_actual` recorriendo el diccionario con un ciclo, para que siga funcionando si más adelante se agrega una variable al modelo.
8. Construir casos de prueba que verifiquen no solo cada regla por separado, sino la dependencia del pasado: repetir la misma percepción dos veces y comprobar que la segunda produce un resultado distinto de la primera. Si produce el mismo, el estado interno no se está actualizando o no se está consultando.

4 Criterio de elección y errores frecuentes

Con ambas plantillas a la vista puede precisarse el criterio anunciado en la Sección 1. La pregunta que resuelve la clasificación es la siguiente: ¿puede la misma percepción exigir acciones distintas en momentos distintos? Si la respuesta es no —si a cada percepción le corresponde siempre la misma acción— el ejercicio es de agente reactivo simple, por más variables que tenga la percepción. Si la respuesta es sí, el ejercicio requiere un modelo, porque lo que rompe la correspondencia fija entre percepción y acción es exactamente la información del pasado.

Conviene notar que el criterio no depende del tamaño del problema. Un agente reactivo simple puede tener seis entradas y doce reglas y seguir siendo reactivo simple; un agente basado en modelo puede tener una sola entrada y tres reglas. Lo que decide es la dependencia del pasado, no la complejidad aparente.

4.1 Errores frecuentes

Observación 4.1 (Usar un diccionario no convierte a un agente en basado en modelo). Almacenar la percepción actual en un diccionario, o guardar constantes de configuración en él, no crea estado interno. El estado interno existe cuando las acciones del agente lo modifican y esas modificaciones afectan decisiones posteriores. Si al eliminar el diccionario el comportamiento del agente no cambia, no había modelo.

Observación 4.2 (Reglas que no se excluyen). Escribir las reglas como `if` independientes en lugar de encadenarlas permite que dos ramas se ejecuten ante una misma percepción. En el agente reactivo simple el `return` suele enmascarar el problema; en el agente basado en modelo no hay `return` que lo enmascare, y el resultado es un modelo actualizado dos veces por una sola percepción.

Observación 4.3 (Duplicar información derivable). Mantener a la vez una capacidad total y un contador de espacio ocupado obliga a actualizar dos variables en cada acción, y basta olvidar una para que el modelo quede inconsistente. La regla es guardar la fuente de verdad y calcular el resto cuando se necesite.

Observación 4.4 (Probar cada regla en aislamiento). Un caso de prueba que satisface la condición de la regla k pero también la de una regla anterior no prueba la regla k : prueba la anterior. Cada caso debe construirse de modo que la regla que se quiere verificar sea la primera cuya condición se cumple.

4.2 Síntesis

Las dos plantillas comparten la función de percepción y la traducción de las reglas del enunciado a una cadena ordenada de condiciones excluyentes. Se diferencian en que el agente basado en modelo antepone una estructura de estado, delega sus modificaciones en funciones auxiliares y consulta ese estado dentro de las condiciones. Instanciar cualquiera de las dos consiste, en ambos casos, en leer

el enunciado buscando tres cosas: qué observa el agente, qué debe recordar —si es que debe recordar algo— y en qué orden decide.

Referencias

- [1] S. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 4.^a ed., Pearson, 2020.